

dbt Core Commands & Components Reference

A quick-reference guide mapping dbt transformation workflow components to CLI execution commands.

dbt Architecture Overview

dbt (data build tool) enables data teams to transform raw data inside data warehouses (like Snowflake) using modular SQL and Jinja templating. This reference sheet covers project initialization, transformation models, seeds, snapshots, quality testing, macros, and documentation commands.

1. Quick Reference: Components vs. CLI Commands

dbt Component	Primary CLI Command	Key Function & Purpose
Project Setup	<code>dbt init</code> / <code>dbt debug</code>	Initializes project structure and tests database connections.
Models	<code>dbt run</code>	Executes SQL transformations (Views, Tables, Incremental).
Seeds	<code>dbt seed</code>	Loads local static CSV files into database tables.
Snapshots	<code>dbt snapshot</code>	Captures historical data changes over time (SCD Type 2).
Tests	<code>dbt test</code>	Runs generic schema checks (unique, not_null) and custom SQL tests.
Packages	<code>dbt deps</code>	Downloads external packages defined in packages.yml (e.g., dbt_utils).
Analysis	<code>dbt compile</code>	Compiles Jinja SQL into dry-run executable warehouse SQL.
Docs & Lineage	<code>dbt docs generate</code>	Generates interactive documentation portal and visual DAG graph.
Unified Build	<code>dbt build</code>	Executes seeds, snapshots, models, and tests in dependency order.

2. Detailed Command Reference by Category

Category 1: Project Setup & Connection Testing

`$ dbt init`

Description: Initializes a new dbt project folder with standard subdirectories (models, seeds, tests, macros, snapshots). Prompts interactively to configure connection profiles (profiles.yml) for Snowflake, BigQuery, Postgres, etc.

\$ dbt debug

Description: Tests your database connection parameters, credentials, environment variables, and verify profile configurations in profiles.yml.

Category 2: Transformation Models & Materializations**\$ dbt run**

Description: Compiles Jinja SQL and executes all .sql models inside the models/ directory, building physical tables, views, or incremental models in Snowflake.

\$ dbt run --select

Description: Runs a specific model or subfolder (e.g., dbt run --select fct_ratings or dbt run -m staging.netflix).

\$ dbt run --select ++

Description: Graph selector syntax: +model_name runs the target model and all upstream dependencies; model_name+ runs the target model and all downstream children.

Category 3: Seeds (Static CSV Uploads)**\$ dbt seed**

Description: Loads static local CSV files located in the seeds/ folder (e.g., seed_movie_release_dates.csv) directly into warehouse tables. Ideal for lookup tables, country mappings, or zip codes.

Category 4: Snapshots (SCD Type 2 History Tracking)**\$ dbt snapshot**

Description: Executes snapshot scripts stored in the snapshots/ directory. Captures point-in-time changes for slowly changing dimensions by adding dbt_valid_from and dbt_valid_to timestamps.

† Category 5: Data Quality Testing**\$ dbt test**

Description: Runs all generic tests defined in schema.yml (such as unique, not_null, relationships, and accepted_values) as well as singular SQL test scripts in tests/.

\$ dbt test --select

Description: Restricts execution to tests configured specifically for the target model or source.

Category 6: Package Dependencies & Dry-Run Analysis

\$ dbt deps

Description: Reads packages.yml and downloads third-party packages (e.g., dbt-labs/dbt_utils or calogica/dbt_expectations) into the project.

\$ dbt compile

Description: Compiles all Jinja models, macros, and analysis files (in analysis/) into raw SQL scripts inside target/compiled/ without executing queries against Snowflake.

Category 7: Documentation Portal & Lineage DAG**\$ dbt docs generate**

Description: Parses project metadata, database information schema, and YAML descriptions to generate catalog.json and manifest.json.

\$ dbt docs serve

Description: Launches a local web server (default port 8080) displaying the interactive documentation portal and the visual Directed Acyclic Graph (DAG) showing full source-to-mart data lineage.

Category 8: Unified Production Pipeline (`dbt build`)**\$ dbt build**

Description: Runs seeds, snapshots, models, and tests in their exact topological graph dependency order. If a seed or upstream test fails, downstream execution for dependent models is automatically halted.

Production Workflow Best Practices

1. **Use Proportional Execution:** In development, run `dbt run --select state:modified+` to only build changed models.
2. **Enforce Tests in CI/CD:** Always include `dbt test` or `dbt build` in pull request validation pipelines.
3. **Document as You Code:** Define model and column descriptions in `schema.yml` alongside generic tests.